

M09 / STUDY COMPANION

M09 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
DE09-01 ETL vs ELT and transformation boundaries	3
DE09-02 Raw, staging, clean/intermediate and serving contracts	3
DE09-03 Deterministic transformations and production SQL style	3
DE09-04 Deduplication and key integrity	3
DE09-05 Schema checks and data contracts	3
DE09-06 Source-to-target reconciliation	3
DE09-07 Incremental processing and safe reruns	3
DE09-08 Failure behavior, retries, quarantine and atomic state	3
DE09-09 Lineage, run metadata and auditability	4
DE09-10 Build and defend a reliable batch pipeline	4
DE09-11 dbt-style project anatomy, sources and dependency graph	4
DE09-12 Executable data tests, quality severity and CI boundaries	4
DE09-13 Incremental models, materialization, documentation and handoff	4

Read this first

RECONSTRUCT FIRST / CHECK SECOND

Standalone Zero-to-Job-Ready Data Engineer 2026

Use this after you have studied M09. Try to reconstruct each idea before reading the bullets.

DE09-01 | ETL vs ELT and transformation boundaries

ETL transforms before target load; ELT loads raw/near-raw first then transforms in the analytical platform. Preserve evidence either way.

DE09-02 | Raw, staging, clean/intermediate and serving contracts

Raw preserves arrival; staging enforces source shape/types; intermediate owns reusable logic; serving exposes a stable downstream contract.

DE09-03 | Deterministic transformations and production SQL style

Determinism needs explicit grain/types/rounding and trustworthy tie rules; same business input should not change because row order or clock changed.

DE09-04 | Deduplication and key integrity

Dedup starts with business identity and version precedence. Ambiguous same-timestamp conflicts are quarantined, not guessed.

Numbers worth reconstructing, not memorizing:

- Duplicate fixture: 4 source = 1 staged + 2 quarantined + 1 discarded; serving=1700.00.

DE09-05 | Schema checks and data contracts

Contract = fields/types/required/null/time/allowed values/version/failure behavior. Preserve raw before rejecting drift.

DE09-06 | Source-to-target reconciliation

Reconcile counts + keys + important measures. Batch001=6 rows/22930.00; after batch002=9 rows/84870.00.

Numbers worth reconstructing, not memorizing:

- Batch001 serving: 6 / 22930.00. After batch002: 9 / 84870.00. Increase=61940.00.

DE09-07 | Incremental processing and safe reruns

Incremental = selection boundary + idempotent target write + atomic state + late/change strategy. Replay must not multiply results.

DE09-08 | Failure behavior, retries, quarantine and atomic state

Retry transient failures, not deterministic bad data. Quarantine with reasons. Advance state only after target success.

Numbers worth reconstructing, not memorizing:

- Bad batch: 5 source, 1 staged, 4 quarantined, ratio .80 -> fail. Lock case succeeds on attempt 2.

DE09-09 | Lineage, run metadata and auditability

Lineage explains origin/dependencies; run metadata explains what executed, on which source, with which counts/results/failures.

DE09-10 | Build and defend a reliable batch pipeline

Integrated loop: evidence -> contract -> transform -> identity -> reconcile -> quality gate -> commit -> state -> audit -> safe rerun.

Numbers worth reconstructing, not memorizing:

- Fresh retest in clean runtime: 3 rows / 61980.00.

DE09-11 | dbt-style project anatomy, sources and dependency graph

dbt: source() anchors raw/external input; ref() connects models; project config/materializations shape the DAG/runtime.

DE09-12 | Executable data tests, quality severity and CI boundaries

Data tests are executable assertions. Generic tests protect common rules; singular tests return project-specific violations.

DE09-13 | Incremental models, materialization, documentation and handoff

Incremental materialization is a correctness contract. $\geq$ max timestamp handles boundary ties here, not arbitrary late history; docs/tests/runbook complete handoff.